WeGo-Robotics

MUJOCO SIMULATOR · CLOUD GPU

로봇 없이 수집부터 추론까지

PIPER Studio 시뮬레이터 — 고성능 GPU와 로봇없이 데이터 수집부터 추론까지 해보자!

1부 가상 환경에서 한 바퀴
가상 팔·카메라로 수집 · 학습 · 추론

2부 클라우드 GPU로 학습
로컬 GPU 없이 학습 환경 구성

ⓒ 2026 WeGo-Robotics. 본 자료의 무단 복사·배포·유출은 법적으로 보호받지 못하며, 관련 법령에 따라 책임이 따를 수 있습니다.

PART 01

1부

가상 환경에서 한 바퀴

MuJoCo 가상 팔·카메라로 수집 → 학습 → 추론

시뮬레이터 켜기 | 가상 로봇 연결 | 가상 카메라 등록 | 가상환경 배치 | 조종 | 데이터 수집 | 학습 | 추론

WeGo-Robotics

로봇이 없으면 여기서 막힙니다

교육·해커톤 현장에서 실습을 시작할 때 겪는 세 가지 어려움

01 로봇 순번 대기

팀은 여럿, 팔은 한두 대
내 차례까지 손을 놓고 기다리게 됩니다

02 조종할 리더암이 부족

사람 시연을 기록하려면 팔로워 외에 조종용 리더암이 한 대 더 필요합니다

03 망가뜨릴까 조심조심

초보자의 첫 조작은 충돌·과부하가 걱정돼
과감하게 연습하기 어렵습니다

시뮬레이터가 대신하는 것

MuJoCo 가상 팔 = 실물 팔 · 키보드·마우스 = 리더암 · 렌더 카메라 = USB 웹캠

실제와 동일한 것

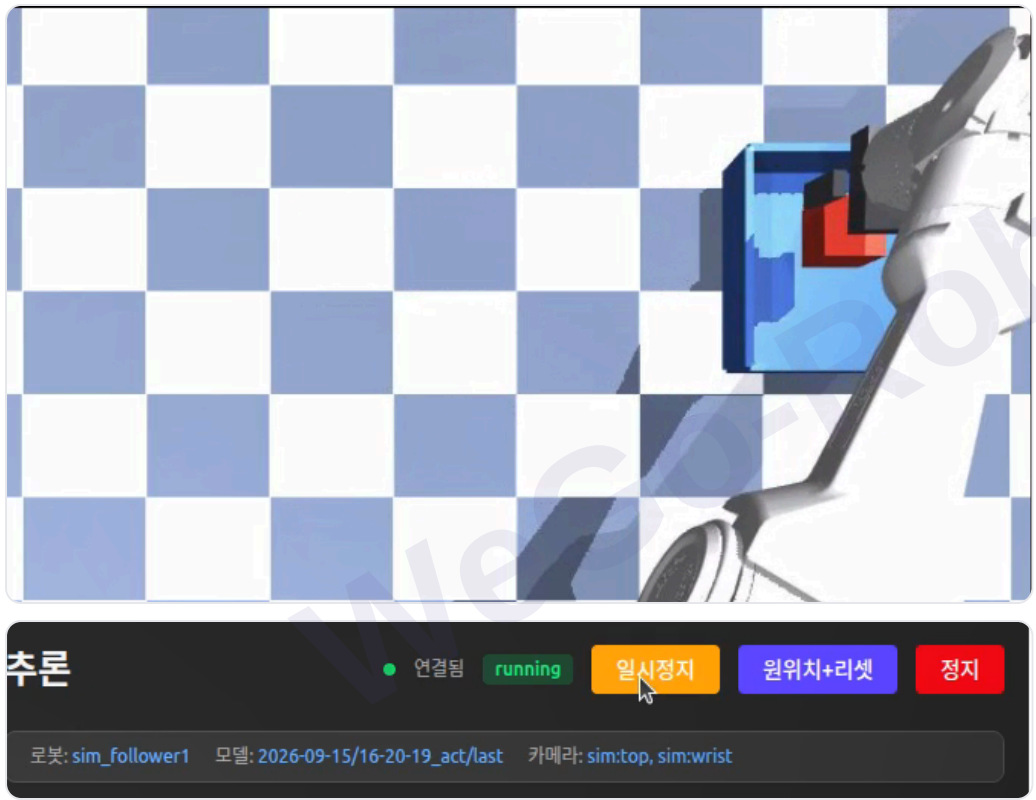
PIPER Studio 화면 · LeRobot 데이터셋 포맷 · 학습·추론 절차 · CLI 명령어

장비가 준비되기를 기다리는 대신, **오늘 바로 수집부터 추론까지 실습합니다**

WeGo-Robotics

먼저 결과부터 — 학습한 모델로 가상 로봇을 움직입니다

이 영상에서 만들 것 · 빨간 큐브를 집어 파란 상자에 넣는 정책 — 수집부터 추론까지 전부 가상 환경에서



과제
Pick the cube and put in the box — 큐브를 집어 상자에 담기

데이터
사람이 직접 조종해 만든 25 에피소드 · 10,403 프레임

모델
ACT · 카메라 2대(탑뷰·손목) · 관절 7차원

결과
학습한 모델로 가상 팔을 움직여 큐브에 접근

사용한 실물 장비 — 로봇 0대 · 리더암 0대 · 카메라 0대. 학습용 GPU는 클라우드로도 충분

WeGo-Robotics

전체 흐름 — 여덟 단계

준비 세 단계 → 무대 짜기 → 조종 익히기 → 수집 → 학습 → 추론. 준비는 처음 한 번뿐이고, 그 다음부터는 수집·학습·추론만 반복합니다

준비 — 처음 한 번

1. 설정 › 서비스에서 piper-simd 켜기
2. 로봇 › 포트에서 sim_follower1 연결
3. 카메라 › 스캔 후 가상 카메라 등록

무대 — 과제 정하기

4. 가상환경에서 물체 배치
   무엇을 집어 어디에 넣을지 정합니다 — 기본 환경을 그대로 써도 됩니다

조종 — 손에 익히기

5. 조종 창에서 마우스·키보드로 팔 조작
   그리퍼를 움직이고 여닫는 방법부터, 환경을 리셋하는 방법까지

본 작업 — 계속 반복

6. 수집 — 에피소드 녹화
7. 학습 — ACT 등 정책 학습
8. 추론 — 시뮬레이터에서 정책 검증

수집·학습·추론의 기본 절차는 동일
가상 장치를 연결한 뒤에는 같은 화면에서 수집·학습·추론을 진행합니다

망쳐도 괜찮습니다
가상 환경은 바로 초기화 가능 · 충돌해도 손상 없음 — 마음껏 반복하세요

WeGo-Robotics

왜 화면이 똑같은가 — 시뮬레이터도 '하나의 로봇'입니다

시뮬레이터(`simd`)는 실물 로봇·카메라 데몬과 같은 방식으로 데이터를 내보내는 또 하나의 장치입니다

실물이라면

로봇 데몬이 팔 상태를,
카메라 데몬이 영상을 내보냅니다

시뮬레이터는

`simd` 하나가 MuJoCo 세계를 돌리며
가상 팔 + 가상 카메라를 같은 방식으로 내보냅니다

그래서

수집·학습·추론·E-STOP 화면과 절차가
실물과 완전히 동일합니다

이름으로 구분합니다

가상 장치에는 `sim_` 접두사(`sim_follower1`)와 보라색 SIM 배지가 붙습니다 — 실물과 섞여 있어도 헷갈리지 않습니다

목적을 분명히 하고 시작합니다

시뮬레이터는 절차 연습과 설정 비교(A/B)용입니다 — 여기서 나온 성공률이 실물 성능을 보장하지는 않습니다

앞으로 나올 모든 화면은 **실물 로봇을 쓸 때 그대로**입니다 — 시뮬에서 익힌 순서가 실물에서 재사용됩니다

WeGo-Robotics

화면 지도 — 왼쪽 메뉴만 알면 길을 잃지 않습니다

여덟 단계가 각각 어느 메뉴에서 일어나는지 — 이 지도를 기억해 두세요



1. 설정 › 서비스 — 시뮬레이터 켜기
2. 로봇 — 가상 팔 연결
3. 카메라 — 가상 카메라 등록
4. 가상환경 — 물체 배치
5. 조종 창 (수집 화면에서 팝업)
6. 수집 — 에피소드 녹화
7. 학습 — 정책 학습
8. 추론 — 정책 검증

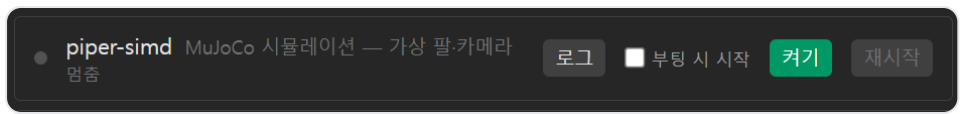
우측 하단 **E-STOP**은 모든 화면에 떠 있습니다 — 시뮬에서도 같은 자리에서 같은 방식으로 동작합니다

STEP 1 / 8 · 준비

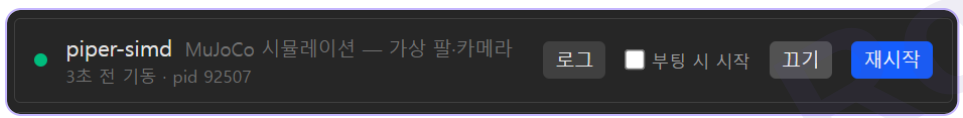
시뮬레이터 켜기 — 설정 › 서비스

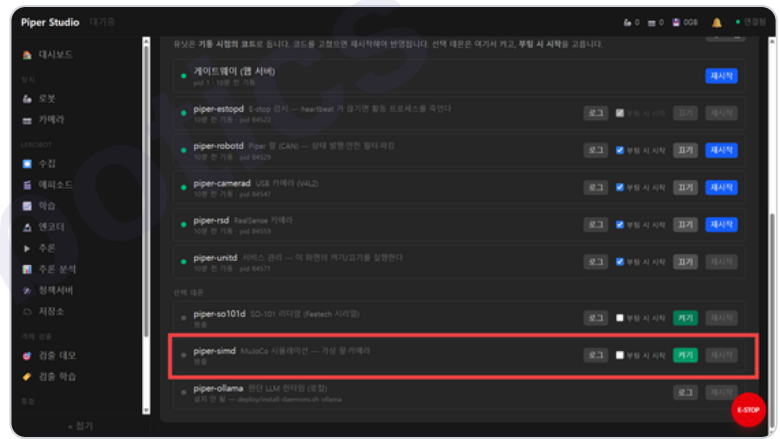
시뮬레이터는 선택 데몬입니다. 평소엔 꺼져 있고, 필요할 때 이 화면에서 켭니다

1 켜기 전 — 회색 점 · 멈춤 · 버튼은 [켜기]



2 [켜기] 누른 뒤 — 초록 점 + pid · 버튼이 [끄기]·[재시작]으로





화면 위치

위치
설정 › 서비스 탭 › 맨 아래 선택 데몬 그룹

부팅 시 시작
계속 쓸 거면 체크 — 다음 부팅부터 자동으로 실행됩니다

실행되지 않으면 — 원인은 보통 둘
`mujoco` 없음 → simd가 안 뜸 · `libEGL` 없음 → 팔은 돌지만 카메라가 빈 화면 · [로그]에서 확인

WeGo-Robotics

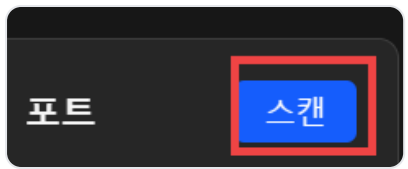
WeGo-Robotics

STEP 2 / 8 · 준비

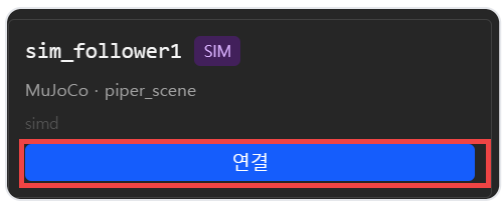
가상 로봇 연결 — 로봇 › 디바이스

누르는 버튼은 두 개입니다 — 빨간 박스를 따라 [스캔] → [연결]

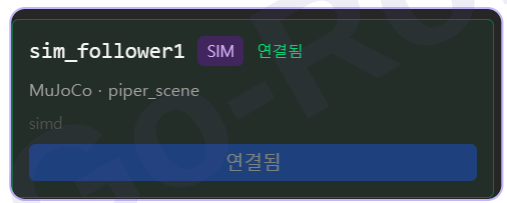
1 오른쪽 위 [스캔] — 포트에 가상 팔이 나타납니다



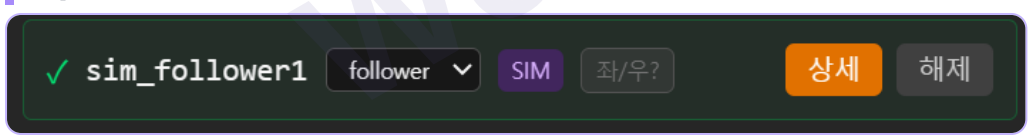
2 카드의 [연결]을 누릅니다

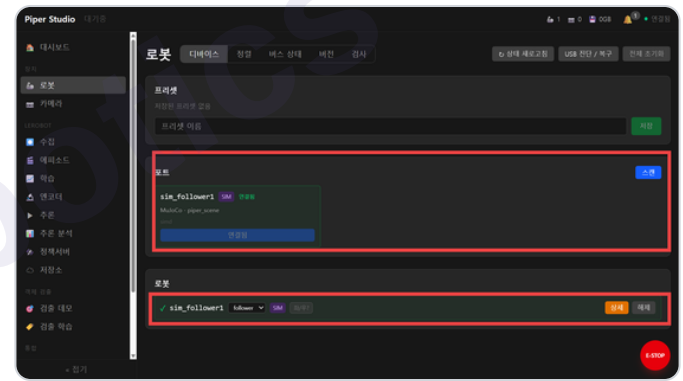


3 '연결됨'으로 바뀌면 성공



연결되면 아래 '로봇' 목록에 `follower` 역할로 등록 · 상단 로봇 개수 0 → 1 · [상세] 관절 상태 · [해제] 연결 끊기





화면 위치

SIM 배지
이 배지가 붙은 장치는 모두 가상 장치

piper_scene
팔 + 작업대 + 큐브 + 상자가 들어 있는 기본 장면

스캔해도 안 나오면
STEP 1의 piper-simd가 초록 점인지 확인 → [스캔] 다시

STEP 3 / 8 · 준비

가상 카메라 등록 — 카메라 › 장치

가상 환경 안에 카메라 세 대가 들어 있습니다. 실물 웹캠과 같은 목록에 표시됩니다



sim:top
위에서 내려다본 작업대 — 큐브와 상자 위치가 한눈에 들어옵니다

sim:front
정면 — 팔 전체 자세를 확인하는 용도

sim:wrist
손목 장착 — 그리퍼와 물체의 위치를 가까이서 확인

등록 후 사용 가능
미등록은 찾기만 한 상태 — 이름을 넣고 [등록]해야 녹화·추론에서 선택 가능

맨 왼쪽 /dev/video0은 실제 USB 웹캠입니다 — 실물 카메라와 가상 카메라가 같은 화면에서 함께 관리됩니다

WeGo-Robotics

STEP 3 / 8 · 준비

등록하고 나면 — 실물 카메라와 같은 화면

EV · 노출 · Gain · 포화까지 동일하게 계측됩니다 — 실물로 넘어갔을 때 이 화면이 낯설지 않도록



사용 가능 (2)
등록되면 위쪽 사용 가능 그룹에 표시됩니다 — 여기 있는 카메라만 녹화·추론에서 쓸 수 있습니다

밝기를 숫자로
Sim top은 +1.9 EV · 포화 36.6%, Sim wrist는 +0.9 EV · 포화 10.6% — 시점마다 밝기가 다르다는 것도 수치로 보입니다

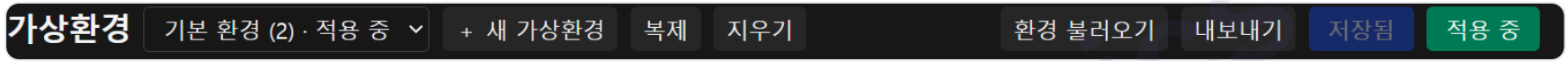
고정된 조명 조건
조명이 변하지 않으니 프로파일 보정이 필요 없습니다 · 같은 조명 조건으로 반복 가능

연습도 됩니다 — 노출·게인·화이트밸런스가 실물 카메라와 같은 이름으로 동작해서, 조명 감시·프로파일·회색 카드 절차를 시뮬에서 미리 연습할 수 있습니다

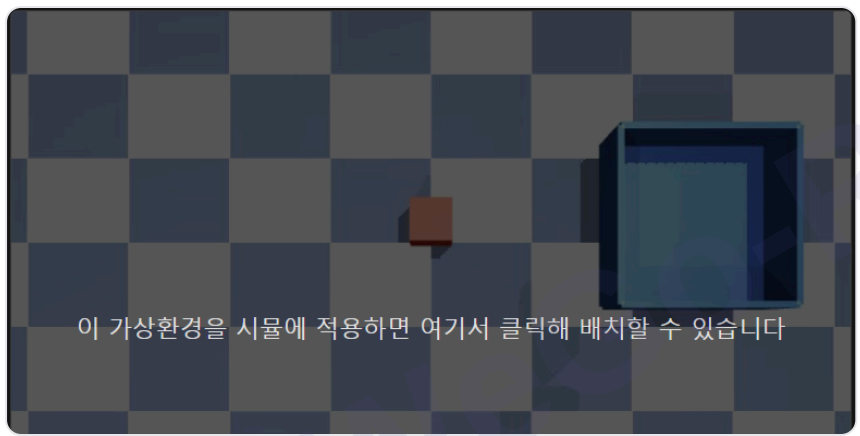
WeGo-Robotics

가상환경 — 과제를 직접 짭니다

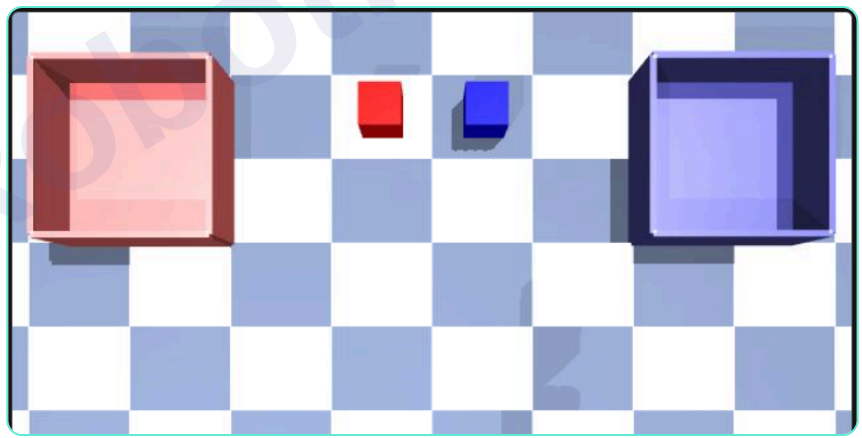
사이드바 장치 › 가상환경 — 여기서 무엇을 집어 어디에 넣을지를 정합니다



1 적용 전 — 탑뷰는 아직 이전 환경을 보여줍니다



2 [시뮬에 적용] 후 — 탑뷰를 클릭해 배치



물체 추가
상자 · 구 · 원기둥 · 캡슐 · 타원체 · 통 (조립)

환경 단위로 저장
드롭다운으로 전환 · 복제 · 내보내기/불러오기로 팀원과 공유

배치는 클릭 한 번
목록에서 물체를 고르고 탑뷰를 클릭하면 그 자리로 · 좌표도 같이 기록됩니다

[시뮬에 적용]을 잊지 마세요
적용해야 조종·수집에 반영됩니다 — 적용되면 적용 중으로 바뀝니다

WeGo-Robotics

STEP 4 / 8 · 무대

물체 속성 — 잡을 수 있게 만들려면



과제를 바꿀 수 있습니다 — 큐브 두 개와 통 두 개를 색깔 맞춰 놓으면 색 분류 과제가 됩니다 (오른쪽 화면). 조종·수집·학습 절차는 그대로입니다

움직임 (잡을 수 있다) — 이 체크가 핵심
꺼 두면 바닥에 붙은 고정물이 됩니다. 집을 물건은 반드시 켜고 질량·마찰을 함께 정하세요

기본값이 타입마다 다릅니다
상자·구는 기본 움직임 / 통(조립)은 기본 고정물 — 통이 안 움직이면 이 체크부터 확인

크기는 반지름·반변입니다
한 변 4cm 상자면 0.02 — 처음에 가장 많이 틀리는 값입니다

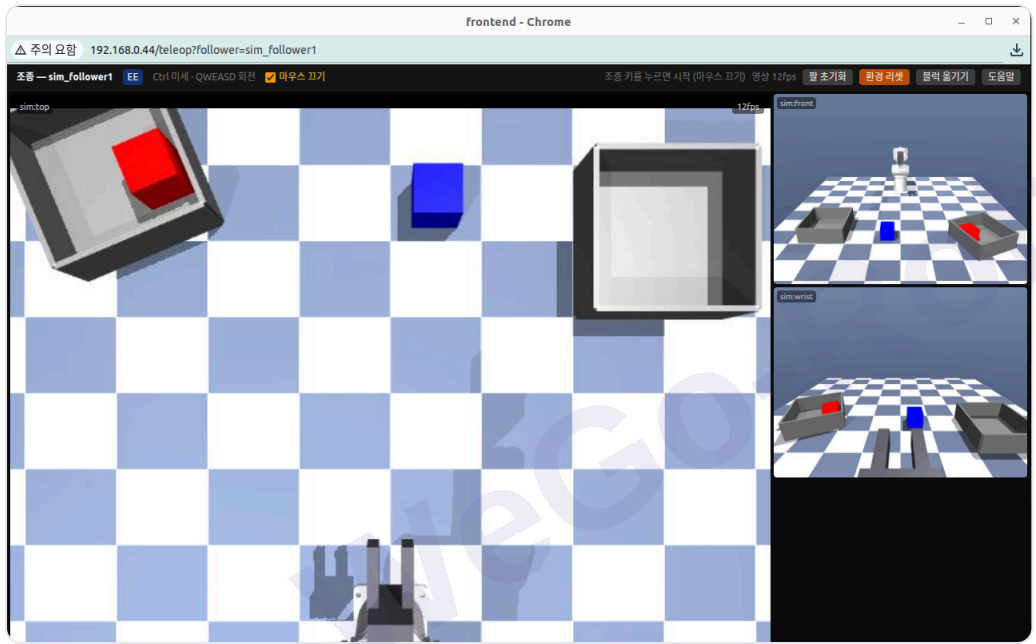
타입마다 다른 항목
상자는 반변 X·Y·Z / 통은 `inner`·`wall_t`·`wall_h` — 통은 담는 물체를 만드는 프리셋



STEP 5 / 8 · 조종

조종 창 — 세 개의 시점을 동시에 보면서

주소는 `/teleop?follower=sim_follower1` · 수집 화면의 [조종 창 열기] 버튼으로도 열립니다



큰 화면 — sim:top
작업대 전체. 큐브와 통, 그리퍼 위치를 여기서 판단합니다

오른쪽 — sim:front · sim:wrist
정면에서 팔 자세를, 손목에서 대상물과의 거리를 확인

위 툴바
EE 표시(말단 조종 전용) · 마우스 끄기 체크 시 키보드로만 조종 · 팔 초기화 · 환경 리셋 · 블럭 옮기기 · 도움말 버튼

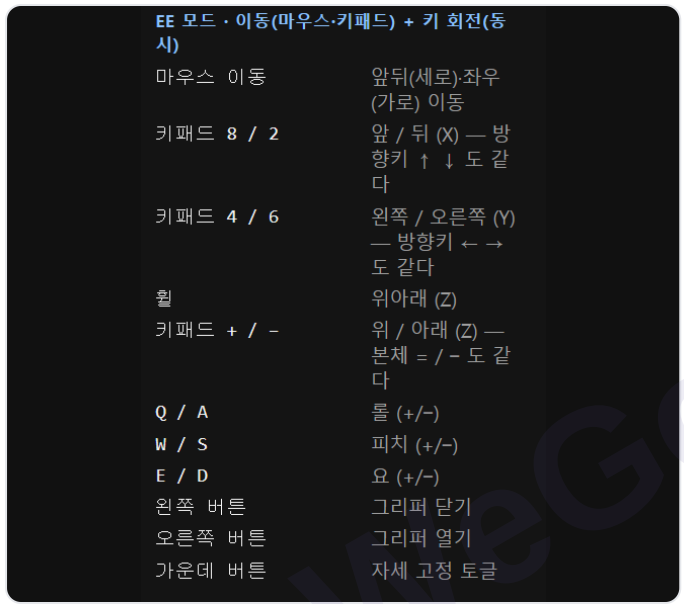
시작하려면 — 화면을 한 번 클릭하면 포인터가 잠기고 조종이 시작됩니다. Esc로 해제

WeGo-Robotics

STEP 5 / 8 · 조종

조작법 — 그리퍼의 위치와 각도를 조절합니다

그리퍼의 목표 위치를 지정하면 각 관절의 움직임을 자동으로 계산합니다



조종 창에서 ? 키 → 도움말

위치 — 어디로 갈 것인가

마우스 이동 = 앞뒤·좌우 (키패드 8·2 / 4·6, 방향키도 동일)
휠 또는 키패드 +·− = 위아래(Z)

자세 — 어떤 각도로 잡을 것인가

Q·A Roll · W·S Pitch · E·D Yaw
위치 이동과 동시에 누를 수 있습니다 — 움기면서 각도를 맞추세요

그리퍼 — 마우스 버튼 또는 키보드

왼쪽 버튼·[닫기 · 오른쪽 버튼·] 열기 · 가운데 버튼 자세 고정 토글
0 키는 토글 — 한 번 누르면 2초에 걸쳐 천천히 여닫습니다

Roll·Pitch·Yaw를 과하게 돌리지 마세요

각도가 과하면 팔이 갈 수 있는 범위가 급격히 줄어듭니다 — 갑자기 움직이지 않으면 [팔 초기화]로 원점에 되돌린 뒤, 각도를 적당히 주고 다시 움직여 보세요

자세 고정을 켜두면 각도가 흔들리지 않아 위에서 아래로 내려 집는 동작이 훨씬 안정적입니다

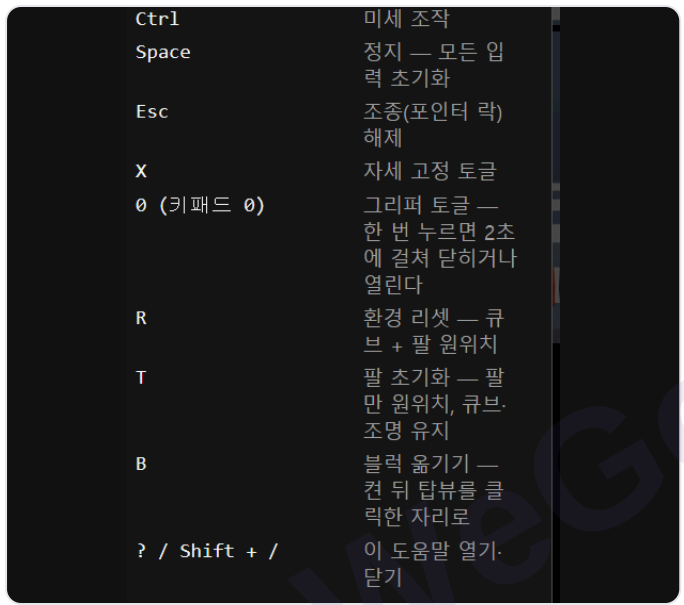
WeGo-Robotics

WeGo-Robotics

STEP 5 / 8 · 조종

데이터 수집에 자주 쓰는 단축키

자주 쓰는 단축키를 익히면 다음 에피소드를 빠르게 준비할 수 있습니다



R · 환경 리셋 — 가장 많이 씁니다

물체와 팔을 모두 원위치로. 한 에피소드가 끝나면 이 키로 다음 에피소드를 시작합니다

B · 블럭 옮기기 — 데이터를 다양하게

누른 뒤 ① 옮길 물체 클릭 → ② 놓을 자리 클릭 — 두 번 클릭입니다.
같은 자리만 반복하면 그 자리만 배웁니다 — 매 에피소드 위치를 바꾸세요
과제 구성 자체를 바꾸려면 STEP 4 가상환경에서

T · 팔 초기화 — 큐브는 그대로

팔만 원위치. 큐브 위치와 조명은 유지되므로 같은 배치로 다시 찍을 때 씁니다

속도 조절

`Shift` 빠르게 · `Ctrl` 미세 조작

급할 때

`Space` 정지 — 모든 입력 초기화

권장 루틴 — 에피소드 종료 → `R`로 리셋 → `B`로 큐브 위치 변경 → 다음 녹화

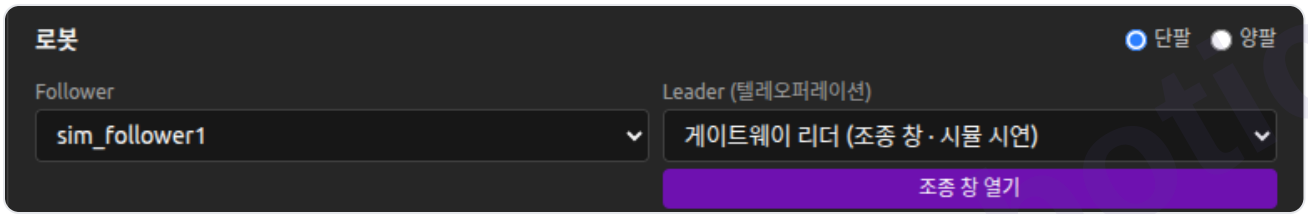
WeGo-Robotics

WeGo-Robotics

STEP 6 / 8 · 수집

수집 설정 ① 로봇 — 리더암 자리에 '게이트웨이 리더'

수집 화면에서 가장 중요한 한 줄 — Leader를 '게이트웨이 리더 (조종 창 · 시뮬 시연)'로



Follower
sim_follower1 — 실제로 움직이며 데이터를 남기는 팔

Leader (텔레오퍼레이션)
게이트웨이 리더 — 리더암 대신 조종 창의 키보드·마우스 입력을 받습니다

[조종 창 열기]
이 버튼이 앞에서 본 조종 창을 띄웁니다

이 조합이 핵심입니다 — 실물 수집에서는 Leader에 리더암 포트를 지정합니다. 시뮬레이터에서는 그 자리에 게이트웨이 리더를 넣어서, 실물 장비 없이 같은 수집 절차를 진행합니다

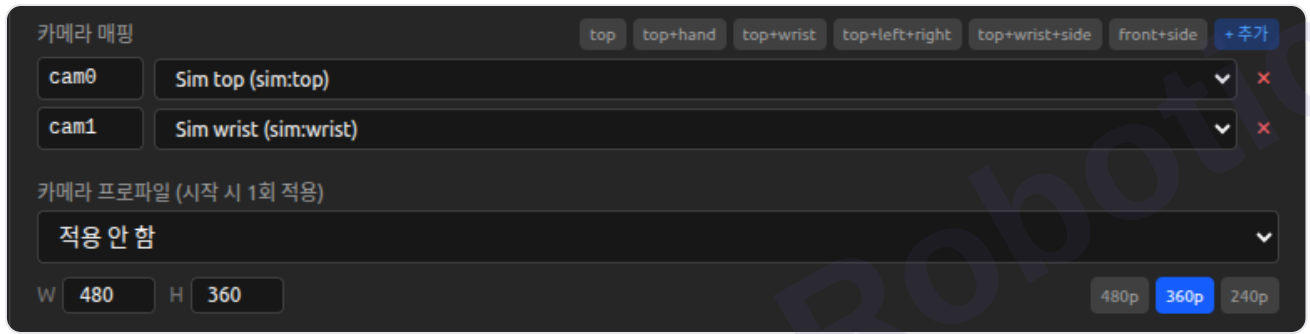
WeGo-Robotics

WeGo-Robotics

STEP 6 / 8 · 수집

수집 설정 ② 카메라 매핑 — 이름을 여기서 확정합니다

cam0 · cam1이라는 이름이 학습과 추론까지 그대로 따라갑니다



프리셋 칩
top · top+hand · top+wrist · front+side ... 자주 쓰는 조합을 한 번에

이 예시의 조합
cam0 ← Sim top (전체 상황)
cam1 ← Sim wrist (그리퍼 주변)

해상도 480×360
360p면 충분합니다 — 높이면 프레임을 놓치기 쉽습니다

이름을 기억하세요
추론할 때 cam0·cam1에 같은 카메라를 다시 연결해야 정책이 동작합니다

카메라 프로파일은 적용 안 함으로 둡니다 — 시뮬레이터는 조명이 고정이라 보정이 필요 없습니다

WeGo-Robotics

WeGo-Robotics

수집 설정 ③ 데이터셋 — 수집 횟수와 녹화 시간



Task 설명 — 수행할 작업을 적는 문장

`Pick the cube and put in the box`

영어로, 동작을 그대로 서술하는 짧은 한 문장으로 씁니다

에피소드 25 · FPS 15

가상 환경의 집기·옮기기 과제는 20~30 에피소드면 첫 결과를 볼 수 있습니다. 15fps면 충분합니다

에피소드 60초 · 리셋 60초

한 에피소드를 녹화할 시간과, 다음 에피소드를 준비할 시간.
익숙해지면 둘 다 30초 정도로 줄여도 됩니다

Hub 업로드

체크하면 수집이 끝나는 대로 Hugging Face에 업로드됩니다

음성 안내

브라우저가 단계를 읽어줍니다 — 화면을 안 봐도 진행 파악

화면 아래에는 현재 설정을 반영한 실행 명령어가 그대로 보입니다 — 같은 작업을 터미널에서도 실행할 수 있습니다

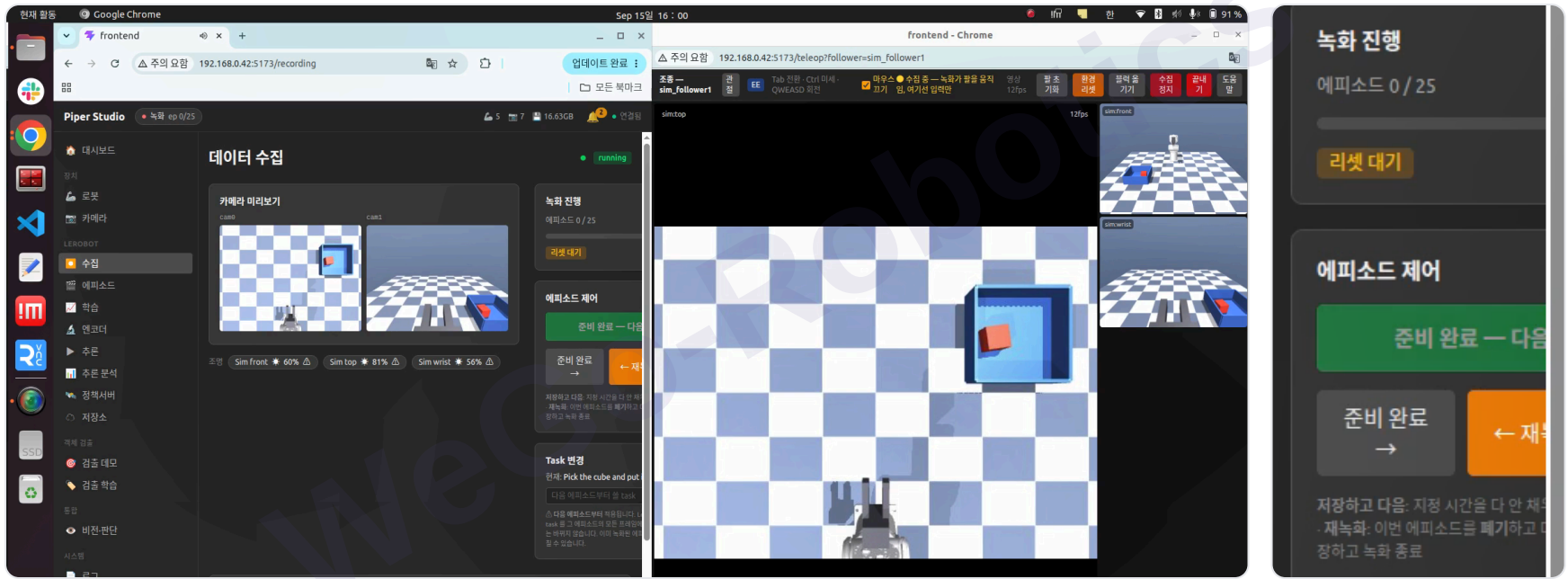
```
/home/sw-han/miniconda3/bin/python3 -u /home/sw-han/wego-piper-web/wrapper/start_record.py --robot.type=piper_follower_shm --robot.port=sim_follower1 --robot.cameras={"cam0":
{"type":"shm","segment":"sim_top","width":480,"height":360,"fps":15},"cam1":{"type":"shm","segment":"sim_wrist","width":480,"height":360,"fps":15}} --teleop.type=piper_leader_shm --teleop.port=web_leader1
dataset.repo_id=wego-hansu/sim_data2 --dataset.single_task=Pick the cube and put in the box --dataset.num_episodes=25 --dataset.fps=15 --dataset.episode_time_s=60 --dataset.reset_time_s=60 --
dataset.streaming_encoding=true --dataset.vcodec=auto --dataset.encoder_threads=4 --dataset.encoder_queue_maxsize=100 --dataset.push_to_hub=true --play_sounds=false
```

WeGo-Robotics

STEP 6 / 8 · 수집

녹화 중 — 창 두 개를 나란히 놓고

왼쪽에서 진행 상황을 보고, 오른쪽에서 조종합니다 — 모니터가 하나면 좌우 분할을 권합니다



녹화 진행
에피소드 0 / 25 · 지금이 리셋 대기인지 녹화 중인지 표시

[준비 완료 → 다음]
리셋 시간을 다 안 써도 바로 다음 에피소드로 넘어갑니다

[← 재녹화]
방금 녹화한 에피소드를 폐기하고 다시 — 실패했으면 망설이지 말고 누르세요

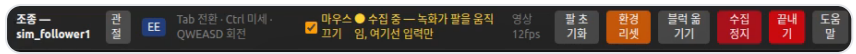
Task 변경
다음 에피소드부터 적용 — 이미 찍은 것은 바뀌지 않습니다

WeGo-Robotics

STEP 6 / 8 · 수집

녹화 중에 놓치기 쉬운 것 두 가지

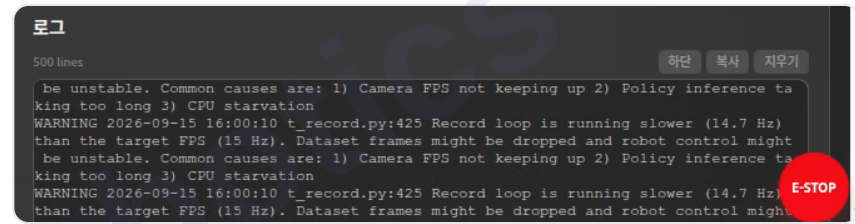
1 조종 창 툴바가 수집 모드로 바뀝니다



"녹화가 팔을 움직임, 여기선 입력만"
수집 프로그램이 팔을 제어합니다. 조종 창은 입력을 넘겨주는 역할만 합니다

[수집 정지] · [끝내기]
녹화 중에만 나타나는 버튼 — 조종 창에서도 수집을 멈출 수 있습니다

2 로그에 뜨는 FPS 경고



Record loop is running slower
목표 15Hz인데 실제 14.7Hz — 프레임이 빠질 수 있다는 경고입니다

조금 느린 건 괜찮습니다
0.3Hz 차이는 넘어가도 됩니다. 차이가 크게 벌어지면 해상도를 낮추거나 인코더 스레드를 늘리세요

수집 루틴 — 녹화 시작 → 큐브 집어 상자에 넣기 → [준비 완료 → 다음] → B로 큐브 위치 변경 → 반복. 실패한 에피소드는 [← 재녹화]

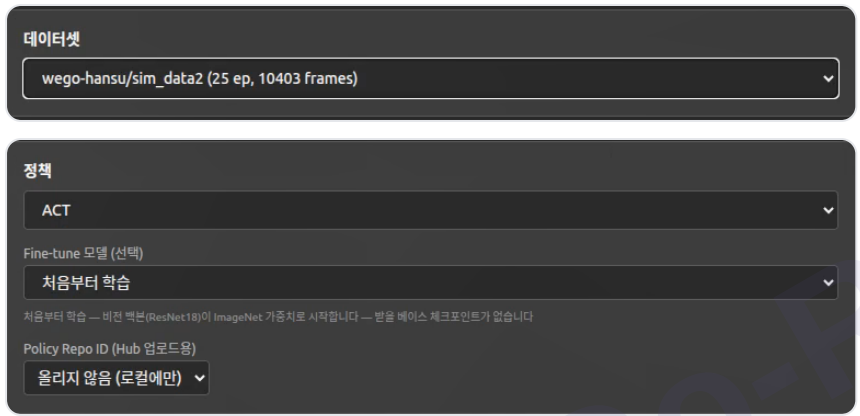
WeGo-Robotics

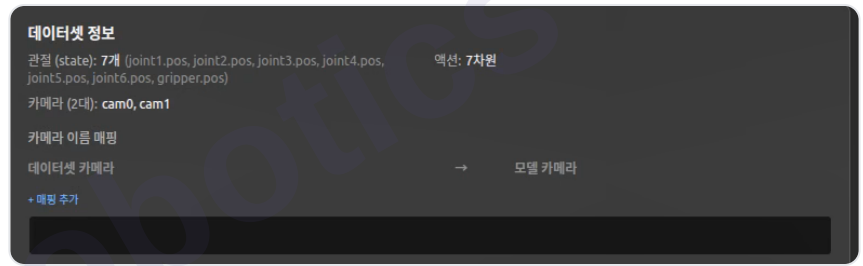
WeGo-Robotics

STEP 7 / 8 · 학습

학습 — 데이터셋만 고르면 나머지는 읽어옵니다

수집이 끝나면 학습 화면에서 방금 만든 데이터셋이 목록에 바로 보입니다





25 ep · 10,403 frames
에피소드 하나당 평균 400프레임 남짓 — 15fps로 에피소드당 약 28초입니다

관절 7개 · 액션 7차원
joint1~6 + gripper. 데이터셋에서 자동으로 읽어와 표시합니다

카메라 2대 — cam0, cam1
수집 때 정한 이름이 그대로 넘어왔습니다

정책은 ACT로 · 처음부터 학습을 선택했습니다 — 비전 백본(ResNet18)은 ImageNet 가중치로 시작하고, 나머지는 새로 학습합니다

WeGo-Robotics

STEP 7 / 8 · 학습

학습 파라미터 — 처음이라면 이 값 그대로

ACT 파라미터



chunk_size 100 / n_action_steps 100
한 번 추론할 때 100스텝 분량의 동작을 한꺼번에 예측합니다 — ACT가 매끄럽게 움직이는 이유

첫 학습은 기본값으로
기본값으로 학습하면 나중에 결과를 비교할 기준이 생깁니다

공통 파라미터



Steps 100,000 · Save 20,000
2만 스텝마다 체크포인트 저장 — 학습 도중 저장한 체크포인트로도 추론해 볼 수 있습니다

AMP bf16
RTX 5090 기준 권장값. VRAM 여유가 있으면 Batch를 32~48로 올려도 됩니다

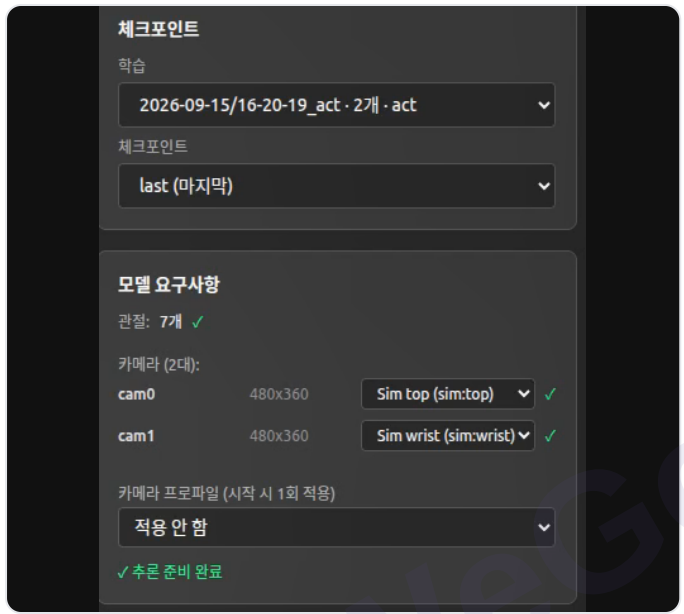
학습은 브라우저를 닫아도 계속됩니다 — 서버에서 실행되는 작업이라 중간에 창을 닫거나 다른 화면으로 옮겨가도 괜찮습니다

WeGo-Robotics

WeGo-Robotics

STEP 8 / 8 · 추론

추론 준비 — 수집 때와 같은 카메라인지 검사합니다



체크포인트 선택

학습 실행을 고르고 그 안에서 체크포인트를 고릅니다. last가 가장 최근 것입니다

모델 요구사항 — 자동 검사

모델이 학습된 조건과 지금 장치가 맞는지 대조합니다.
관절 7개 ✓ · cam0 ← Sim top ✓ · cam1 ← Sim wrist ✓

여기서 막히는 경우가 가장 많습니다

cam0에 다른 카메라를 연결하면 체크가 뜨지 않습니다 — 수집 때와 같은 카메라를 같은 슬롯에

추론 모드

로컬(wrapper) — 같은 장비에서 바로 실행 · 기본값
서버(gRPC) — 학습 서버의 GPU를 원격으로 쓸 때

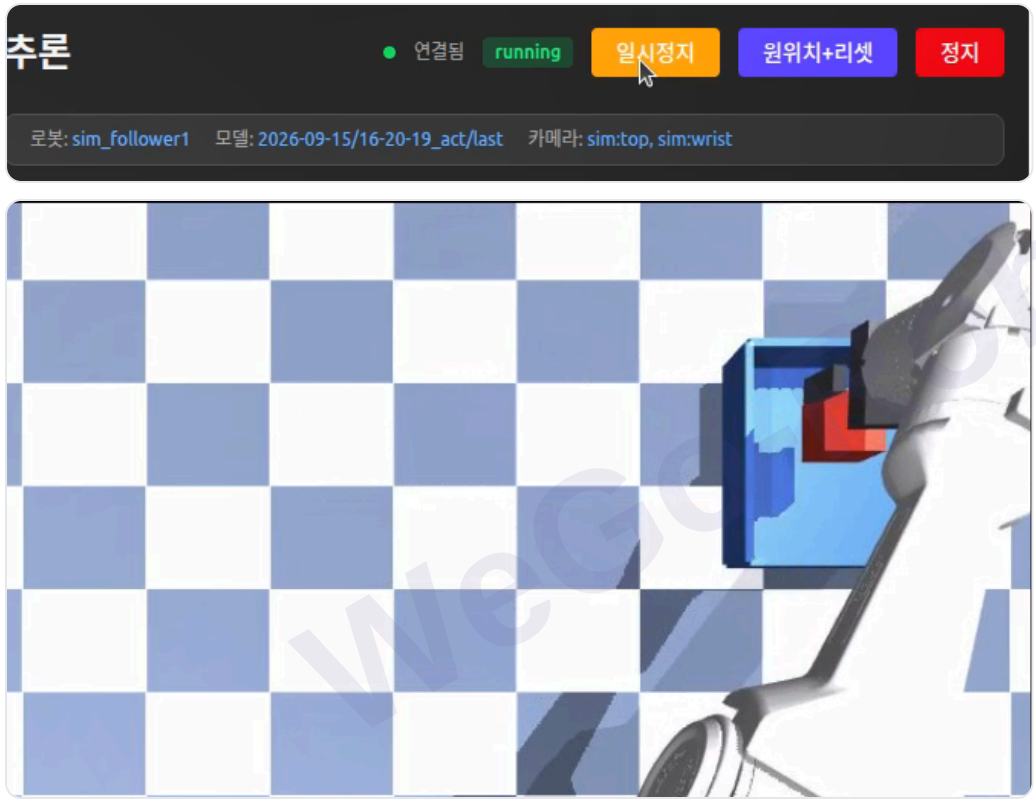
전부 초록 체크가 되면 **추론 준비 완료**가 뜨고 [추론 시작] 버튼을 누를 수 있습니다

WeGo-Robotics

WeGo-Robotics

STEP 8 / 8 · 추론

추론 실행 — 이번엔 정책이 팔을 움직입니다



running · 14.9 fps
모델이 초당 15번 정도 판단하며 팔을 움직이고 있습니다

[원위치+리셋]
한 번의 시도가 끝나면 이 버튼으로 같은 조건에서 다시 — 반복해서 평가할 때 사용합니다

실행 속도 — 처음엔 낮게
관절 속도 제한 36%로 시작 · 동작이 안정적이면 올립니다

평가 방법 — 가상환경 창을 옆에
추론 창 옆에 가상환경 창을 같이 띄우고, 탑뷰에서 큐브를 클릭해 옮기면 돌아가는 중에도 위치가 바뀝니다 — 10회 시도해 성공한 비율을 기록 · 8회 성공 = 80%

잘 안 되면 — 데이터가 부족하거나(에피소드 늘리기), 큐브 위치가 단조로웠거나(B로 다양화), 학습이 덜 됐습니다(스텝 늘리기)

실물 로봇으로 전환할 때 바꿀 설정

시뮬레이터에서 익힌 절차가 그대로 재사용됩니다. 다만 시뮬레이터에서의 성공률이 실물에서 그대로 나오지는 않습니다

바뀌는 것

화면	시뮬레이터	실물
로봇 › 포트	sim_follower1	CAN 포트
수집 › Leader	게이트웨이 리더	리더암 포트
카메라	sim:top / wrist	USB 웹캠
카메라 프로파일	적용 안 함	필수 — 회색 카드 보정

그대로인 것

- 수집·학습·추론 화면 구성
- LeRobot 데이터셋 포맷
- ACT 등 정책·파라미터
- cam0·cam1 매핑 규칙

주의 — Sim-to-Real 격차

실물 환경에서는 모델의 성능이 달라질 수 있습니다
렌더링된 화면과 실제 카메라 영상은 픽셀 분포가 다릅니다 · 마찰·무게·지연도 다릅니다 — 시뮬 성공률은 설정 간 비교(A/B)에 쓰세요

시뮬 안에서도 조건을 지키세요
카메라 위치·해상도를 바꾸면 그 전에 찍은 데이터셋·체크포인트와 관측이 달라집니다 — 바꿨다면 다시 수집부터

실물 실습에도 활용할 수 있는 것
조작 방법 · 데이터 점검 기준 · 학습 설정 · 수집부터 추론까지의 절차

실물 데이터가 생기면
시뮬레이터에서 학습한 체크포인트를 Fine-tune 모델로 지정해 이어서 학습할 수 있습니다

시뮬레이터의 목적은 실물을 대체하는 것이 아니라, 실물 앞에 앉기 전에 절차를 몸에 익히는 것입니다

WeGo-Robotics

자주 걸리는 것 — 증상별 해결법

막히면 여기부터 — 대부분 simd 재시작이나 재연결로 풀립니다

증상	원인 · 조치
서비스에서 simd를 켜도 안 뜹니다	`mujoco` 미설치 — [로그]의 import 오류를 확인하고 설치
카메라 연결 실패 · 빈 화면	`libEGL` 없음 또는 simd 재시작 직후 — 카메라 [스캔]으로 상태를 다시 맞추기
"연결됨"인데 "관절값을 읽지 못했습니다"	simd 재시작으로 연결이 풀린 상태 — 로봇 화면에 다시 들어가 [연결]
"simd가 응답하지 않습니다"	재시작하기 전에 [로그]를 먼저 확인 — 실제 원인이 로그에 남아 있는 경우가 많습니다
팔이 "누가 이미 쥐고 있습니다"로 잠김	이전 프로세스가 남긴 연결 흔적 — simd 재시작으로 정리됩니다
잡은 물체가 자꾸 미끄러집니다	그리퍼가 10° 이상 기울면 놓치기 쉽습니다 — 자세 고정을 켜고 수직으로 내려 집기 · 30°·15mm 어긋남이 현재 한계

공통 1순위 — [로그]
설정 › 서비스의 simd 줄에 [로그] 버튼이 있습니다 — 재시작 전에 원인부터 확인하세요

공통 2순위 — 재시작 후 재연결
simd를 재시작했다면 로봇·카메라 화면에서 [연결]·[스캔]을 다시 — 화면 상태와 실제 상태를 맞추는 절차입니다

WeGo-Robotics

1부 정리

시뮬레이터로 해보실 순서

체크리스트

1. 설정 › 서비스 › piper-simd 켜기
2. 로봇 › 스캔 › sim_follower1 연결
3. 카메라 › 스캔 › sim:top · sim:wrist 등록
4. 가상환경 › 배치 후 [시뮬에 적용]
5. 조종 창에서 10분 연습 — 집어서 상자에 넣기
6. Leader = 게이트웨이 리더 · 20~30 에피소드 수집
7. ACT · 기본 파라미터로 학습
8. 추론 → B로 위치 바꿔 10회 시도 → 성공률 기록

기억해 두실 세 가지

실패한 에피소드는 지우세요
재녹화 버튼 한 번이면 됩니다. 나쁜 데이터는 모델이 그대로 따라 배웁니다

큐브 위치를 매번 바꾸세요
B 키. 같은 자리만 반복하면 그 자리만 배웁니다

카메라 슬롯 이름을 지키세요
`cam0·cam1`이 수집·학습·추론 내내 같아야 합니다

필요한 하드웨어

로봇 0대 · 리더암 0대 · 카메라 0대
실행 환경 준비 후 브라우저로 접속
학습 GPU는 클라우드 이용 가능

PIPER Studio · piper-web v0.5.4 · MuJoCo piper_scene · wego-robotics.com

WeGo-Robotics

PART 02

2부

클라우드 GPU로 학습

로컬 GPU가 없어도 학습을 진행합니다

왜 클라우드인가 | 설정 · 클라우드 | 인스턴스 준비 | 데이터셋 업로드 | 학습 실행 · 모니터링 | 체크포인트 회수 | 비용 · 주의

← 메뉴

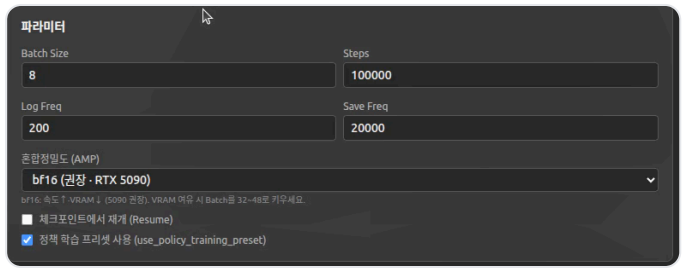
WeGo-Robotics

WeGo-Robotics

2부 · 왜 클라우드인가

로봇은 못 빌려도 GPU는 빌릴 수 있습니다

1부에서 정한 학습 설정 — 이걸 돌리려면 어떤 GPU가 필요한가



1부의 학습 파라미터 — bf16 · RTX 5090 권장이라고 안내됩니다

항목	최소	권장
GPU	컴퓨트 능력 7.5 이상 RTX 20xx · T4	RTX 3090 / 4090 · VRAM 24GB
드라이버	CUDA 13.0 이상 + nvidia-container-toolkit	
안 되는 것	GTX 10xx · Pascal/Volta · Windows/WSL · macOS	

1부는 GPU 없이 됩니다
시뮬레이터·조종·수집은 CPU만으로 동작 — GPU가 필요한 건 학습·추론부터입니다

장비를 새로 사려면
24GB 카드 한 장에 수백만 원 · 실습 한 번 해보려고 들이기엔 부담

빌려 쓰면
이 자료의 예시 학습은 RTX 3060 · 시간당 $0.06 · 20,000스텝에 48분

1부에서 로봇 · 리더암 · 카메라를 화면 안으로 옮겼다면, 2부에서는 마지막 남은 GPU를 시간 단위로 빌립니다

WeGo-Robotics

WeGo-Robotics

2부 · 전체 흐름

여섯 단계 — 마지막 단계를 꼭 기억하세요

GPU 시장 Vast.ai를 PIPER Studio가 직접 호출합니다. 계정과 결제는 Vast.ai 쪽, 나머지 조작은 전부 PIPER Studio 화면에서

Vast.ai 에서 · 처음 한 번

1. API 키 만들기
2. 크레딧 충전

PIPER Studio 에서 · 처음 한 번

3. 설정 › 클라우드에서 키 연결
   API 키 붙여넣기 + SSH 공개키 등록

학습할 때마다 · 반복

4. 기계 빌리기
5. 그 기계에서 학습 실행
6. 다 쓰면 파기

4번과 5번은 별개입니다

빌리기 버튼은 기계만 만듭니다. 학습은 학습 페이지에서 따로 시작합니다

6번을 잊으면 계속 과금됩니다

학습이 끝나도 기계는 자동으로 꺼지지 않습니다

데이터는 어떻게?

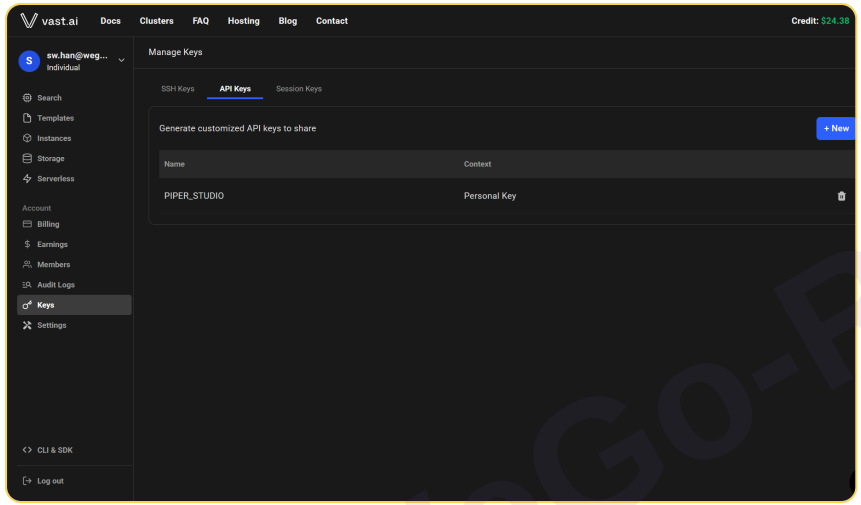
데이터셋은 Hugging Face Hub를 거칩니다 — 1부에서 Hub 업로드를 켜둔 이유

WeGo-Robotics

두 화면을 오갑니다 — Vast.ai 와 PIPER Studio

왼쪽 사이트에서 계정·결제를, 오른쪽 화면에서 나머지 전부를 합니다

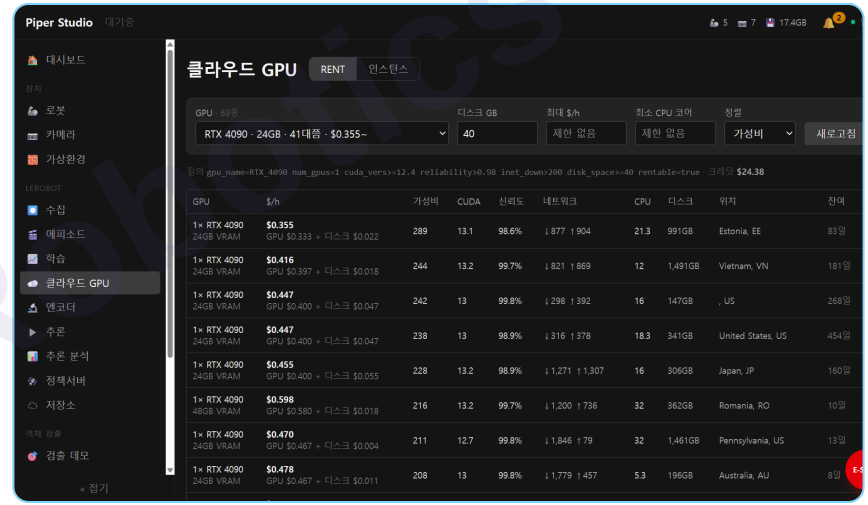
vast.ai — 브라우저로 접속



- Keys — API 키 만들기 (STEP 1)
- Billing — 크레딧 충전 (STEP 2)

여기서 하는 일은 이 두 가지뿐 — 이후에는 들어올 일이 거의 없습니다

PIPER Studio — 익숙한 그 화면



- 설정 › 클라우드 — 키 연결 (STEP 3)
- 클라우드 GPU 메뉴 — 빌리기·인스턴스 관리 (STEP 4·6)
- 학습 — 1부와 같은 화면에서 실행 위치만 클라우드로 (STEP 5)

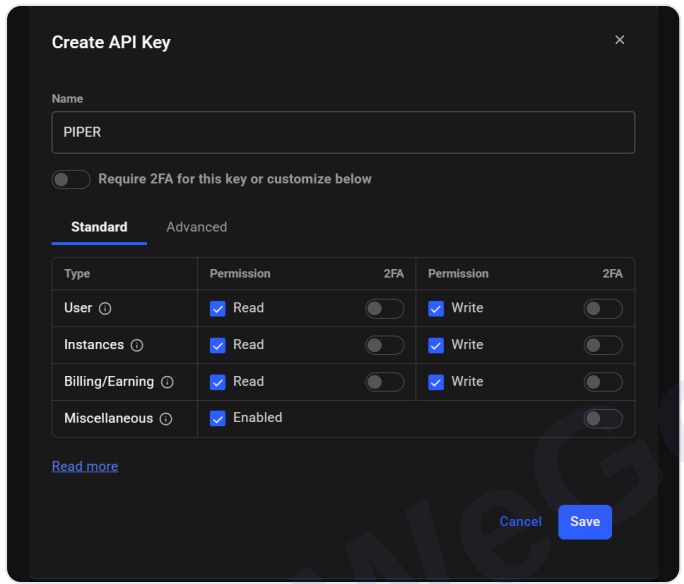
WeGo-Robotics

WeGo-Robotics

STEP 1 / 6 · Vast.ai

API 키 만들기 — 한 번만 보여줍니다

vast.ai 로그인 → 왼쪽 Keys → API Keys 탭 → [+ New]



이름은 알아볼 수 있게

예시에서는 `PIPER` — 나중에 이 키를 지울 때 구분하려는 용도입니다

권한은 기본값 그대로

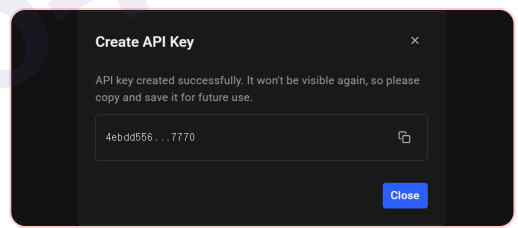
User · Instances · Billing/Earning 의 Read · Write, Miscellaneous Enabled
PIPER Studio가 기계를 빌리고 파기하려면 Instances 쓰기 권한이 필요합니다

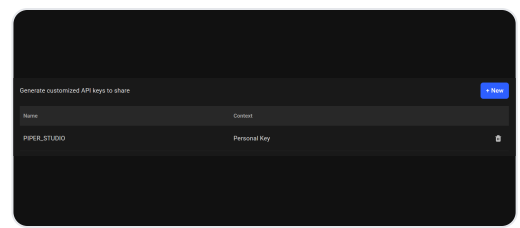
2FA 토글은 꺼 둡니다

켜면 PIPER Studio가 키를 쓸 때마다 인증을 요구하게 됩니다

저장 직후 화면을 닫지 마세요

키 전문은 이때 한 번만 보입니다. 복사 버튼으로 옮겨 두고, 놓쳤으면 지우고 새로 만드세요





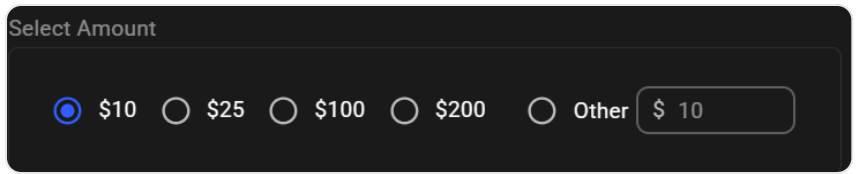
WeGo-Robotics

WeGo-Robotics

STEP 2 / 6 · Vast.ai

크레딧 충전 — 잔액이 0이 되면

Billing › Add Credit

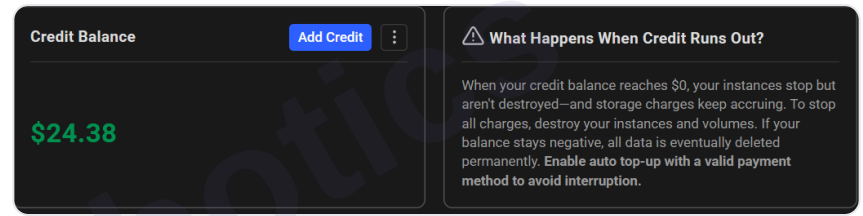


$10부터 충전됩니다
시간당 $0.06짜리 기계라면 165시간을 쓸 수 있는 금액입니다

결제 수단
Stripe(카드) · crypto.com · bitpay — 카드를 등록해두면 Auto Top-Up도 쓸 수 있습니다

쓴 만큼만 차감
충전한 크레딧에서 시간 단위로 빠져나갑니다. 남은 잔액은 환불 가능, 쓴 금액은 환불 불가

Billing — 잔액과 경고



잔액이 0이 되면
인스턴스는 멈추지만 사라지지는 않습니다 — 그리고 디스크 요금은 계속 쌓입니다

마이너스가 유지되면
결국 데이터가 영구 삭제됩니다. 요금을 완전히 멈추려면 인스턴스를 직접 파기해야 합니다

실습 규모라면
이 예시의 전체 학습이 $0.05 수준입니다. $10이면 여러 번 반복해도 남습니다

WeGo-Robotics

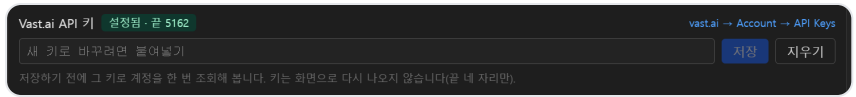
WeGo-Robotics

STEP 3 / 6 · PIPER Studio

키 연결 — 설정 › 클라우드

여기서 하는 일은 두 가지 — API 키 저장과 SSH 공개키 등록

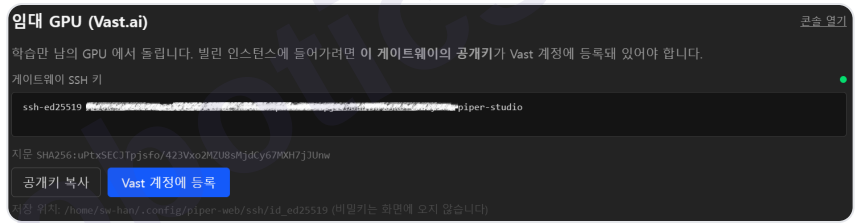
1 Vast.ai API 키 붙여넣기



저장 전에 확인합니다
그 키로 계정을 한 번 조회해 보고 저장합니다

이후에는 끝 네 자리만
키 전문은 화면에 다시 나오지 않습니다 — 설정됨 · 끝 5162

2 게이트웨이 SSH 공개키를 Vast 계정에 등록



[Vast 계정에 등록] 한 번
버튼 한 번으로 공개키가 올라갑니다 — 계정 레벨 키는 새 인스턴스에 자동 추가

비밀키는 나오지 않습니다
~/.config/piper-web/ssh/에 보관 · 화면에는 공개키와 지문만

WeGo-Robotics

WeGo-Robotics

STEP 4 / 6 · 기계 빌리기

클라우드 GPU › RENT — 시장에서 고르기

사이드바에 새로 생긴 클라우드 GPU 메뉴 · 탭은 RENT(빌리기)와 인스턴스(빌린 것 관리)

GPU · 69종 | RTX 4090 · 24GB · 41대쯤 · $0.355~ | 디스크 GB: 40 | 최대 $/h: 제한 없음 | 최소 CPU 코어: 제한 없음 | 정렬: 가성비 | 새로고침

GPU	$/h	가성비	CUDA	신뢰도	네트워크	CPU	디스크	위치	잔여
1× RTX 4090 24GB VRAM	**$0.355** GPU $0.333 + 디스크 $0.022	289	13.1	98.6%	↓877 ↑904	21.3	991GB	Estonia, EE	83일
1× RTX 4090 24GB VRAM	**$0.416** GPU $0.397 + 디스크 $0.018	244	13.2	99.7%	↓821 ↑869	12	1,491GB	Vietnam, VN	181일
1× RTX 4090 24GB VRAM	**$0.447** GPU $0.400 + 디스크 $0.047	242	13	99.8%	↓298 ↑392	16	147GB	, US	268일

GPU 드롭다운
`RTX 4090 · 24GB · 41대쯤 · $0.355~`
종류마다 재고와 최저가가 같이 보입니다

디스크 GB
40GB면 충분 — 데이터셋과 체크포인트가 들어갈 자리입니다

$/h 는 합계
GPU 요금 + 디스크 요금으로 나뉘어 표시됩니다

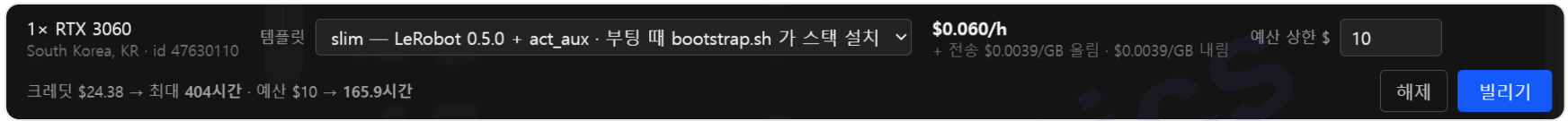
신뢰도 · 위치 · 잔여
신뢰도는 높을수록 좋고, 위치는 전송 속도와 관계됩니다. 잔여는 그 기계가 남아 있는 기간

기본 정렬은 **가성비** — 위쪽에 있는 것을 고르면 대체로 무난합니다. 화면 위의 질의문은 지금 조건을 그대로 보여주는 것이라 참고만 하시면 됩니다

WeGo-Robotics

STEP 4 / 6 · 기계 빌리기

고르면 아래에 — 예산이 시간으로 환산됩니다



템플릿
`slim — LeRobot 0.5.0 + act_aux`
부팅할 때 학습 스택이 자동 설치됩니다 — 직접 설치할 것이 없습니다

시간당 요금
$0.060/h + 전송비 올림·내림 각 `$0.0039/GB`

예산 상한 $
여기 넣은 금액이 바로 시간으로 환산돼 아래에 표시됩니다

크레딧 $24.38 → 최대 404시간
예산 $10 → 165.9시간 · 얼마나 쓸 수 있는지 미리 보입니다

이 예시의 실제 비용

RTX 3060 · $0.060/h · 20,000스텝
약 48분 학습 → **$0.05 남짓**

기계를 고르는 기준

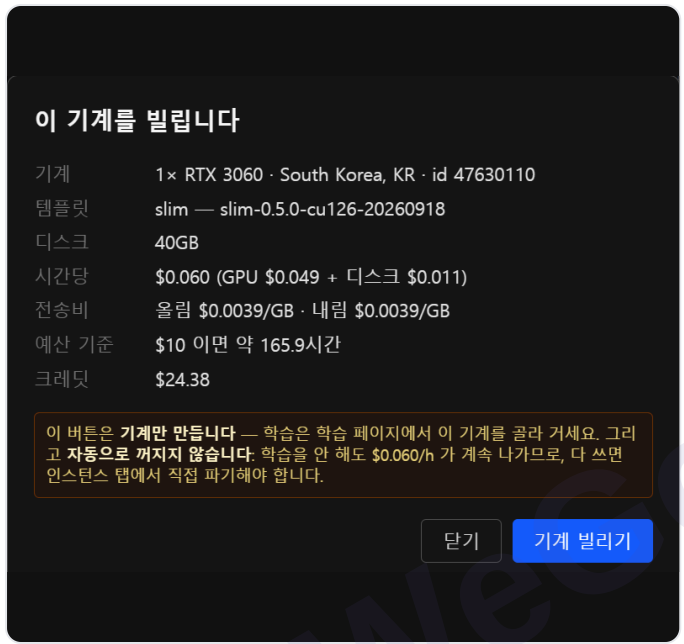
ACT 정도면 VRAM 12GB로 충분합니다
더 큰 모델을 학습할 때 24GB 이상으로 올리세요

WeGo-Robotics

WeGo-Robotics

STEP 4 / 6 · 기계 빌리기

빌리기 를 누르기 전에 — 경고 두 줄



이 버튼은 **기계만 만듭니다** — 학습은 학습 페이지에서 이 기계를 골라 거세요. 그리고 **자동으로 꺼지지 않습니다**: 학습을 안 해도 $0.060/h 가 계속 나가므로, 다 쓰면 인스턴스 탭에서 직접 파기해야 합니다.

① 이 버튼은 기계만 만듭니다

빌렸다고 학습이 시작되지 않습니다. 학습 페이지에서 이 기계를 골라 따로 시작해야 합니다

② 자동으로 꺼지지 않습니다

학습을 안 해도 $0.060/h 가 계속 나갑니다 · 다 쓰면 인스턴스 탭에서 직접 파기

확인 창에 적힌 것

기계 · 템플릿 버전 · 디스크 · 시간당 요금 · 전송비 · 예산 환산 · 남은 크레딧 — 누르기 전 마지막 확인

WeGo-Robotics

WeGo-Robotics

STEP 4 / 6 · 기계 빌리기

인스턴스 탭 — 준비되는 과정을 지켜봅니다

상태가 running이 되고 success가 뜨면 학습을 시작할 수 있습니다

1 loading — 기계는 잡혔고 아직 준비 중

상태	GPU	$/h	라벨	SSH	진행	
loading id 51852218	RTX 3060	**$0.0604**	piper-box-62684	ssh8.vast.ai:12218	—	파기

2 Pulling — 학습용 이미지를 내려받는 중 (몇 분 걸립니다)

상태	GPU	$/h	라벨	SSH	진행	
loading id 51852218	RTX 3060	**$0.0604**	piper-box-62684	ssh8.vast.ai:12218	slim-0.5.0-cu126-20260918: Pulling from wego-robotics/piper-train	파기

3 running · success — 준비 완료

상태	GPU	$/h	라벨	SSH	진행	
running id 51852218	RTX 3060	**$0.0604**	piper-box-62684	ssh8.vast.ai:12218	success, running ghcr.io/wego-robotics/piper-train_slim-0.5.0-cu126-20260918/ssh	파기

라벨
`piper-box-62684` — PIPER Studio가 붙인 이름. 학습 페이지에서 이 이름으로 고릅니다

합계 $/h
위쪽에 인스턴스 1개 · 합계 $0.0604/h — 지금 나가고 있는 금액입니다

[파기] 버튼
여기 있습니다. 학습이 끝나면 이 버튼을 눌러야 요금이 멈춥니다

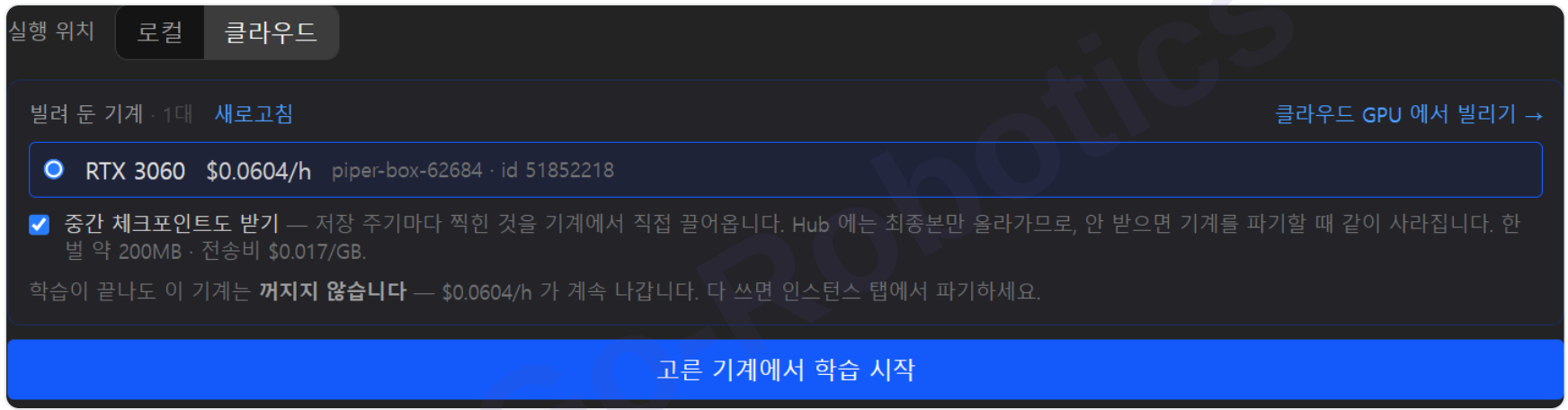
WeGo-Robotics

WeGo-Robotics

STEP 5 / 6 · 학습

1부와 같은 화면 — 토글 하나가 늘었습니다

데이터셋 · 정책 · 파라미터는 1부 그대로 · 아래에 실행 위치 [로컬 | 클라우드] 만 추가



빌려 둔 기계 목록
인스턴스 탭에서 본 그 기계가 여기 나옵니다 · 없으면 [클라우드 GPU에서 빌리기 →]

고르고 시작
라디오로 기계를 선택하고 [고른 기계에서 학습 시작] 한 번

데이터셋은 Hub 경유
1부에서 Hub 업로드를 켜둔 데이터셋을 그 기계가 내려받아 학습합니다

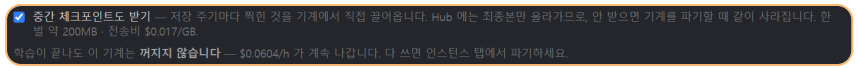
다시 한 번
학습이 끝나도 기계는 꺼지지 않습니다 — 화면에도 같은 안내가 적혀 있습니다

CLI 명령어도 1부와 같은 자리에 그대로 보입니다 — 바뀌는 것은 어느 기계에서 실행하는가뿐입니다

WeGo-Robotics

STEP 5 / 6 · 학습

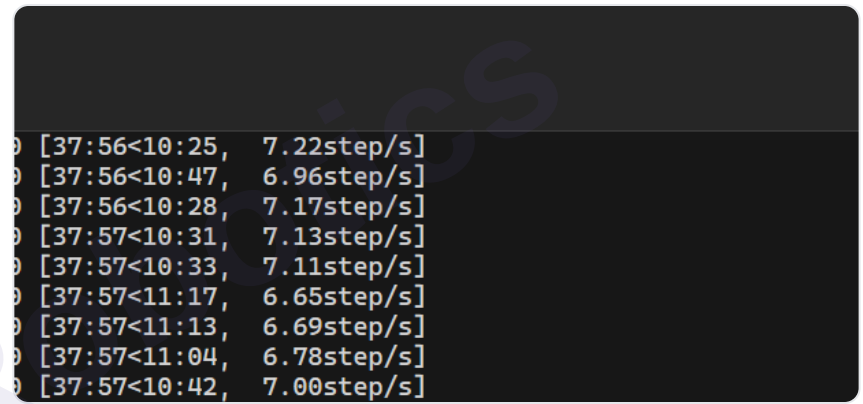
체크포인트 회수 — 이 체크를 켜세요



Hub 에는 최종본만 올라갑니다
중간 체크포인트는 기계 안에만 있습니다 — 안 받으면 파기할 때 같이 사라집니다

켜 두면 저장 주기마다
기계에서 직접 끌어옵니다 · 한 벌 약 **200MB** · 전송비 `$0.017/GB`

왜 중간 것이 필요한가
1부에서 본 것처럼 끝까지 안 가도 중간 체크포인트로 추론해 볼 수 있기 때문입니다



로그는 그대로 흐릅니다
`Training: 77% | 15490/20000 [37:56<10:25, 7.22step/s]`
남은 시간까지 표시되므로 언제 끝날지 가늠할 수 있습니다

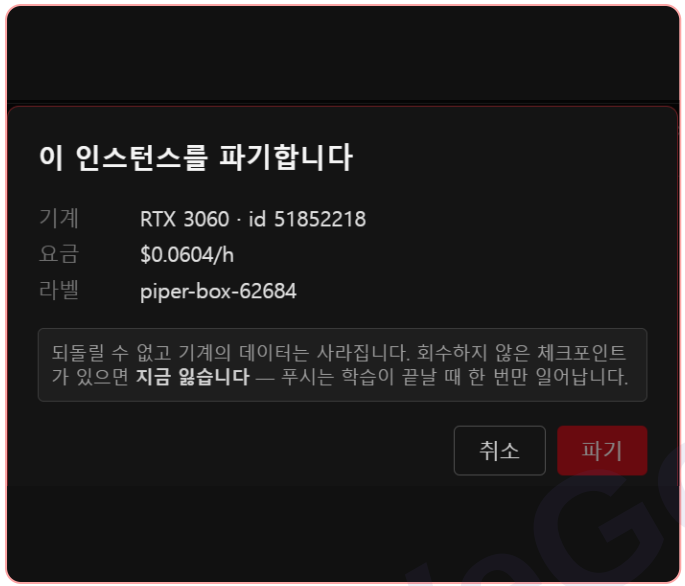
브라우저를 닫아도 됩니다
학습은 빌린 기계에서 돌기 때문에, 창을 닫아도 계속 진행됩니다

WeGo-Robotics

WeGo-Robotics

STEP 6 / 6 · 파기

마지막 단계 — 이걸 안 하면 요금이 계속 나갑니다



파기 전에 확인할 것

회수하지 않은 체크포인트가 있으면 지금 잃습니다 · Hub 푸시는 학습이 끝날 때 한 번만 일어납니다

되돌릴 수 없습니다

기계 안의 데이터는 전부 사라집니다 — 필요한 것을 먼저 내려받고 파기하세요

멈추는 것으로는 부족합니다

크레딧이 0이 되어 인스턴스가 멈춰도 디스크 요금은 계속 쌓입니다. 완전히 멈추려면 파기해야 합니다

학습이 끝나면 이 순서로

① 체크포인트를 받았는지 확인 → ② 추론에 쓸 모델을 저장소에서 확인 → ③ 인스턴스 탭에서 [파기] → ④ 목록이 비었는지 다시 확인

WeGo-Robotics

정리 — 1부와 2부를 합치면

필요한 것과 필요 없는 것

1부가 없앤 것

실물 로봇 · 리더암

실물 카메라

2부가 없앤 것

학습용 GPU 장비

학습 환경 설치

그래도 남는 것

PIPER Studio가 실행되는 PC
시뮬레이터와 화면을 띄우는 환경 · 브라우저로 접속

Vast.ai 계정과 크레딧
$10이면 이 예시 학습을 여러 번 돌릴 수 있습니다

Hugging Face 계정
데이터셋과 모델이 오가는 통로

전체 순서

1. 시뮬 켜기 → 로봇·카메라 연결 → 가상환경 배치 → 조종 연습
2. 게이트웨이 리더로 20~30 에피소드 수집 · Hub 업로드 켜기
3. Vast.ai — API 키 · 크레딧
4. 설정 › 클라우드에서 키 연결
5. 기계 빌리기 → running 확인
6. 학습 › 클라우드 → 중간 체크포인트 받기 체크
7. 인스턴스 파기
8. 추론 → B로 위치 바꿔 10회 → 성공률 기록

기억할 것 세 가지 — 실패한 에피소드는 지우기 · `cam0·cam1` 이름 지키기 · 기계는 직접 파기하기

PIPER Studio · piper-web v0.5.4 · MuJoCo piper_scene · Vast.ai · wego-robotics.com

← 메뉴

ⓒ 2026 WeGo-Robotics. 본 자료의 무단 복사·배포·유출은 법적으로 보호받지 못하며, 관련 법령에 따라 책임이 따를 수 있습니다.